BCA-102
Block I: Mathematical Logic & Discrete Structures (Indepth)

Weightage: 30 Marks Marks

Explain the Syntax and Semantics of Prepositional Logic in depth.

In discrete mathematics, syntax refers to the formal rules that define the structure of logical expressions. It involves a set of symbols: propositional variables (p, q, r) and logical connectives (¬, ∧, ∨, →, ↔). Semantics, on the other hand, deals with the 'truth value' or meaning of these expressions. We evaluate semantics using truth tables to determine if a statement is a Tautology (always true), a Contradiction (always false), or Contingent. Understanding this is vital for designing reliable computer algorithms and circuits.

Mathematical Expression: L = {p, q, r, ...} and C = {¬, ∧, ∨, →, ↔}

What are Normal Forms (CNF and DNF) and their roles in logical simplification?

Normal forms are canonical representations of logical formulas. Conjunctive Normal Form (CNF) is a product of sums (AND of ORs), while Disjunctive Normal Form (DNF) is a sum of products (OR of ANDs). These are used in computer science for 'Resolution'—a proof technique for automated reasoning—and for simplifying Boolean circuits. Every logical formula can be transformed into an equivalent CNF or DNF, which allows for standardized processing by logic gates and theorem provers.

Mathematical Expression: CNF: (p ∨ q) ∧ (¬p ∨ r) | DNF: (p ∧ q) ∨ (¬p ∧ r)

Analyze the Laws of Deduction, specifically the principle of Resolution.

Resolution is a powerful inference rule used for automated theorem proving in both propositional and predicate logic. It operates on the principle that if we have two clauses where one contains a literal 'p' and the other contains its negation '¬p', we can infer a new clause (the resolvent) containing all the other literals. If the repeated application of resolution leads to an empty clause (False), the original set of premises is proven to be inconsistent, thereby proving the theorem by contradiction.

Mathematical Expression: {(A ∨ B), (¬A ∨ C)} ⊢ (B ∨ C)

Describe First-Order Logic (FOL) and its expansion over Propositional Logic.

First-Order Logic (also known as Predicate Logic) is more expressive than Propositional Logic because it uses predicates, constants, variables, and quantifiers. While propositional logic only deals with simple 'true/false' facts, FOL allows us to describe relationships between objects and state properties that apply to all (Universal) or some (Existential) elements in a domain. This is the foundation for database query languages like SQL and artificial intelligence reasoning.

Mathematical Expression: $\forall x\ [P(x) \rightarrow Q(x)]$

Define Universal and Existential Quantifiers with mathematical context.

Quantifiers are used in predicate logic to specify the 'quantity' of elements that satisfy a predicate. The Universal Quantifier (∀) represents 'For All' and is true only if the property holds for every element in the domain. The Existential Quantifier (∃) represents 'There Exists' and is true if there is at least one element for which the property is true. They are critical for formalizing mathematical proofs and software specifications.

Mathematical Expression: $\forall x\ P(x)$ (Universal) | $\exists x\ P(x)$ (Existential)

What is the role of terms in Predicate Logic?

Terms are the 'nouns' of First-Order Logic. They refer to objects in the domain of discourse. A term can be a constant (representing a specific object), a variable (representing an unspecified object), or a function (representing a mapping from objects to other objects). Terms are combined with predicates to form atomic formulas, which are the building blocks of more complex logical statements.

Mathematical Expression: $t = f(t1, t2, ..., tn)$

Explain the fundamental principles of Combinatorics: Permutations.

Permutations deal with the specific 'arrangement' of objects where the order is strictly significant. If you have a set of 'n' items and you want to arrange 'r' of them in a specific sequence, the number of ways to do this is calculated using factorial notation. This is widely used in computer security for calculating the strength of passwords or the number of ways to route data through nodes.

Mathematical Expression: $nPr = n! / (n - r)!$

Explain the fundamental principles of Combinatorics: Combinations.

Combinations involve the 'selection' of objects where the order does not matter. Unlike permutations, selecting items A and B is considered the same as selecting B and A. This is used in probability for determining outcomes in games of chance, selecting committees, or analyzing data samples. The formula includes 'r!' in the denominator to remove the duplicate counts caused by different orderings.

Mathematical Expression: $nCr = n! / [r! * (n - r)!]$

Discuss the Counting and Summation principles in Discrete Structures.

The Sum Rule states that if task A can be done in 'm' ways and task B in 'n' ways (and they are disjoint), then 'A or B' can be done in m+n ways. The Product Rule states that if task A can be done in 'm' ways and task B in 'n' ways, then 'A and B' together can be done in m×n ways. Summation notation (Σ) is then used to express the cumulative total of these sequences, which is fundamental for analyzing the time complexity of loops in programming.

Mathematical Expression: Total Ways = $m \times n$ (Product Rule)

Define Sets and the concept of a Power Set.

A set is a well-defined collection of distinct objects called elements. A Power Set of any set S, denoted P(S), is the set that contains every possible subset of S, including the empty set and S itself. The size of the power set grows exponentially as the number of elements in the original set increases. This is a core concept in data structures for representing groups and combinations.

Mathematical Expression: $|P(S)| = 2^n$

What is the Cartesian Product of sets and its application?

The Cartesian Product (A × B) is the set of all ordered pairs (a, b) where 'a' belongs to A and 'b' belongs to B. In computer science, this is used to describe the set of all possible inputs to a function with multiple parameters or to represent coordinates in a multi-dimensional database or grid system.

Mathematical Expression: $A \times B = \{(a, b) \mid a \in A \land b \in B\}$

Explain the properties of Relations: Reflexive, Symmetric, and Transitive.

A relation R on a set A is: 1. Reflexive if every element relates to itself. 2. Symmetric if whenever a relates to b, then b relates to a. 3. Transitive if whenever a relates to b and b relates to c, then a relates to c. These properties help classify how data points interact—for instance, an 'equals' sign is reflexive, symmetric, and transitive, while a 'greater than' sign is only transitive.

Mathematical Expression: $(aRa) \wedge (aRb \Rightarrow bRa) \wedge ((aRb \wedge bRc) \Rightarrow aRc)$

Define an Equivalence Relation and its impact on set partitioning.

An equivalence relation is a relation that is reflexive, symmetric, and transitive. The significance of an equivalence relation is that it partitions the set into disjoint subsets called 'Equivalence Classes'. Every element belongs to exactly one class, and all elements within a class are 'equivalent' under the relation. This is used in coding theory and database normalization.

Mathematical Expression: Partition $S = E1 \cup E2 \cup \ldots \cup Ek$

What is a Partial Order (Poset) in Discrete Structures?

A Partial Order is a relation that is reflexive, antisymmetric (if aRb and bRa then a=b), and transitive. Unlike an equivalence relation, it describes a 'precedence' or 'hierarchy'. A set with a partial order is called a Poset. Common examples include the 'less than or equal to' relation on numbers or the 'subset' relation on sets.

Mathematical Expression: Reflexive $\wedge$ Antisymmetric $\wedge$ Transitive

Explain the concept of Lattices and their significance.

A Lattice is a special type of Poset where every pair of elements has a unique Least Upper Bound (LUB) and a unique Greatest Lower Bound (GLB). Lattices are the mathematical foundation for Boolean Algebra, which is essential for digital logic design and the operation of logic gates in a CPU.

Mathematical Expression: $\forall a,b \in L: \exists(LUB(a,b) \wedge GLB(a,b))$

Define a Function and its types (Injection and Surjection).

A function is a relation where every input maps to exactly one output. An Injective function (One-to-One) ensures that distinct inputs map to distinct outputs. A Surjective function (Onto) ensures that every possible value in the codomain is mapped to by at least one input. If a function is both, it is a Bijection, meaning every input has exactly one unique output and vice versa.

Mathematical Expression: $f: A \to B$

Describe the Principle of Inclusion-Exclusion.

This principle is a counting technique to find the size of the union of multiple sets. To avoid overcounting elements that appear in the intersection of sets, we add the sizes of individual sets and subtract the sizes of their intersections. For three or more sets, this involves an alternating sequence of adding and subtracting intersections.

Mathematical Expression: $n(A \cup B) = n(A) + n(B) - n(A \cap B)$

Explain the concept of Theorem Proving in discrete mathematics.

Theorem proving is the process of using axioms, definitions, and rules of inference (like Modus Ponens and Resolution) to demonstrate that a specific mathematical statement is logically true. In computer science, automated theorem provers are used to verify the correctness of hardware and critical software systems to ensure they work as intended.

Mathematical Expression: $\text{Premises} \vdash \text{Conclusion}$

What are Normal Forms in the context of Predicate Logic?

Similar to propositional logic, predicate logic uses Prenex Normal Form (PNF) where all quantifiers are moved to the front of the formula. Skolemization is then used to remove existential quantifiers by replacing them with constants or functions. This standardizes formulas for automated processing and mathematical proofs.

Mathematical Expression: $Q1x1\ Q2x2\ \ldots\ Qnxn\ M(x1, \ldots, xn)$

Detail the Syntax of terms in Predicate Logic.

Syntax rules for terms specify how constants, variables, and function symbols can be combined to refer to objects. A term is defined recursively: a variable is a term, a constant is a term, and if 'f' is an n-ary function and t1...tn are terms, then f(t1...tn) is a term. This allows for complex expressions that can represent data in a structured way.

Mathematical Expression: $t ::= c \mid x \mid f(t1, \ldots, tn)$